ECE-251 Final Project Report

Justin Koe

Diego Toribio

Professor Marano

5/13/22

Instruction Set Architecture (ISA)

This single cycle CPU was implemented using the RV32I ISA with modifications to register usage. In this CPU, there are 30 general purpose registers (x1 to x30), x0 is the null register (constant 0) which should never be written to, and x31 is the program counter.

Supported Instructions and Opcodes:

R-type 0110011	**I-type 0010011/0000011**	**S-type 0100011**	**B-type/J-type 1100011**
ADD	ADDI	SW	BEQ
SUB	LW		
OR			
AND			
SLL			
SRL			
XOR			

Instruction Formats:

R-type:

Funct7 (7 bits)	Rs2 (5 bits)	Rs1 (5 bits)	Funct3 (3 bits)	Rd (5 bits)	Opcode (7 bits)

I-type:

IMMediate (12 bits)	Rs1 (5 bits)	Funct3 (3 bits)	Rd (5 bits)	Opcode (7 bits)

S-type:

IMMediate (bits 11:5)	Rs2 (5 bits)	Rs1 (5 bits)	Funct3 (3 bits)	IMMediate (bits 4:0)	Opcode (7 bits)

B-type:

IMMediate (12 bit)	IMMediate (bits 10:5)	Rs2 (5 bits)	Rs1 (5 bits)	Funct3 (3 bits)	IMMediate (bits 4:1)	IMMediate (11th bit)	Opcode (7 bits)

R-type Datapath Diagram

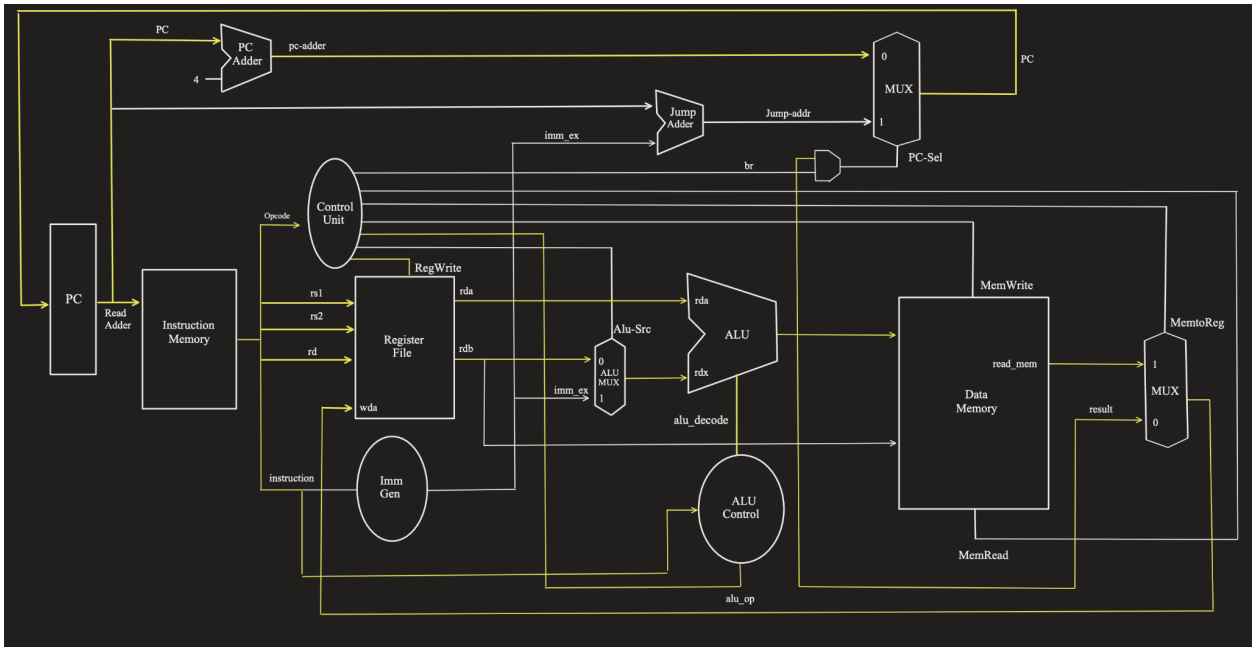


Figure 1: R-Type Datapath

Figure 1 illustrates the datapath of an R-Type instruction which executes arithmetic and logic operations. To start the instruction cycle, the program counter (PC) points to a specific instruction in instruction memory. The instruction is then decoded into the opcode and the three registers (rs1, rs2, rd) that are being used in this instruction.

The opcode enters the control unit where it is decoded into the control lines. These control lines are 1-bit values that are asserted or deasserted based on the opcode. In an R-type instruction, the register write (reg_wrt) control bit is asserted in addition to setting the 4-bit alu operation (alu_op) to indicate the operation the arithmetic logic unit (ALU) should execute on the two operand registers (rs1 and rs2). After the operation is complete, because the reg_wrt control bit is asserted, the result of the ALU is being written to the destination register (rd) in the register file. The instruction cycle ends with the result being written to the register file and on the next rising edge of the clock, the PC is incremented by 4 to point at the next instruction that is stored in memory. The PC is incremented by 4 because the instructions are byte addressable.

I-type Datapath Diagram

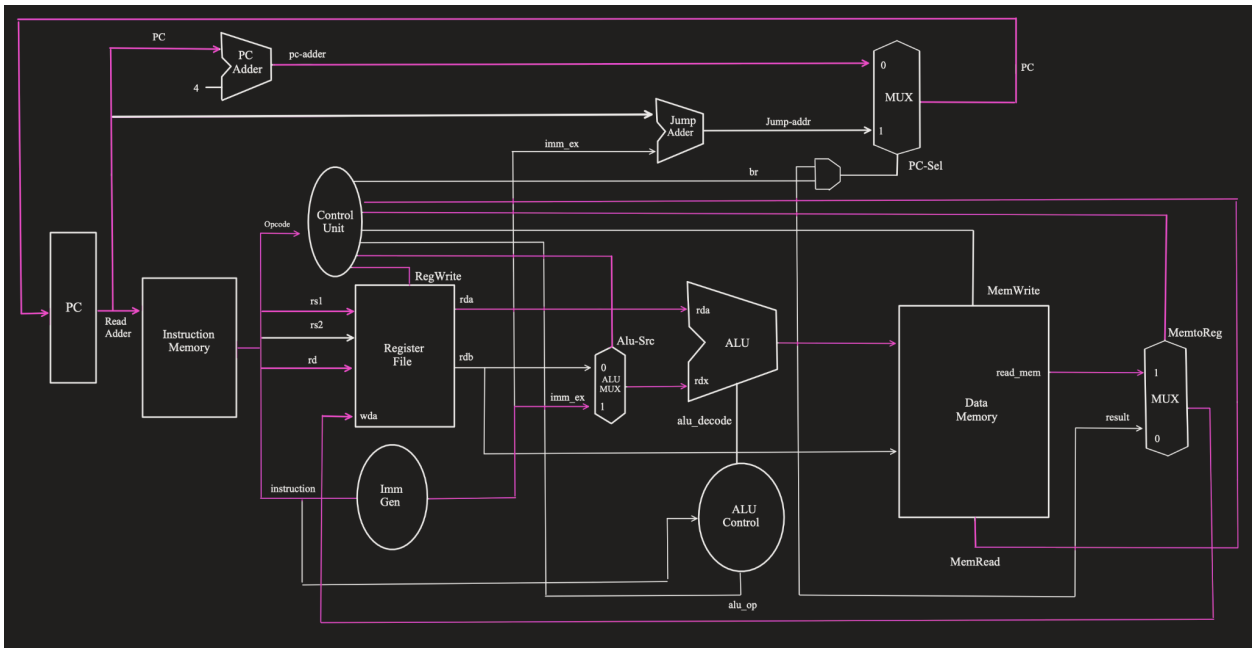


Figure 2: I-Type Datapath

Figure 2 depicts the datapath of an I-type. These types of instructions are used to add immediate values to register values in addition to loading a value from memory into a register. For this walk through, imagine a LW (load word) instruction. The instruction format differs from R-type in that there is an encoded immediate which is decoded and sign extended to 32 bits in the immediate generation module. The start of this instruction is the same as R-type, however the main difference occurs in the control unit. The instruction types (R, I, S, B) are mainly differentiated by their opcode values. Therefore, the control lines are different for every instruction type, which is why in an I-type instruction, the following control lines are asserted: alu_src, mem_to_reg, reg_wrt and mem_rd. The 4-bit alu_op is also defined by this opcode which in the case of a LW instruction is 0.

When alu_src is asserted, the decoded immediate value is used as the second input for the ALU rather than a second register. The immediate value is added to the contents of the first

register, rs1, to compute the memory address that should be read. Then, the mem_to_reg, mem_rd, and and reg_wrt control lines work in unison to allow a value from memory to be read (mem_rd), chosen as the value to be written back (mem_to_reg), and finally written to the indicated register (reg_wrt). Similar to the R-type instruction, the instruction cycle ends when the memory value is written to the register file, and on the next rising edge of the clock, the PC is incremented by 4 to point to the next instruction in memory. The PC is incremented by only 4 because this was an I-type instruction, not a branch type instruction.

S-type Datapath Diagram

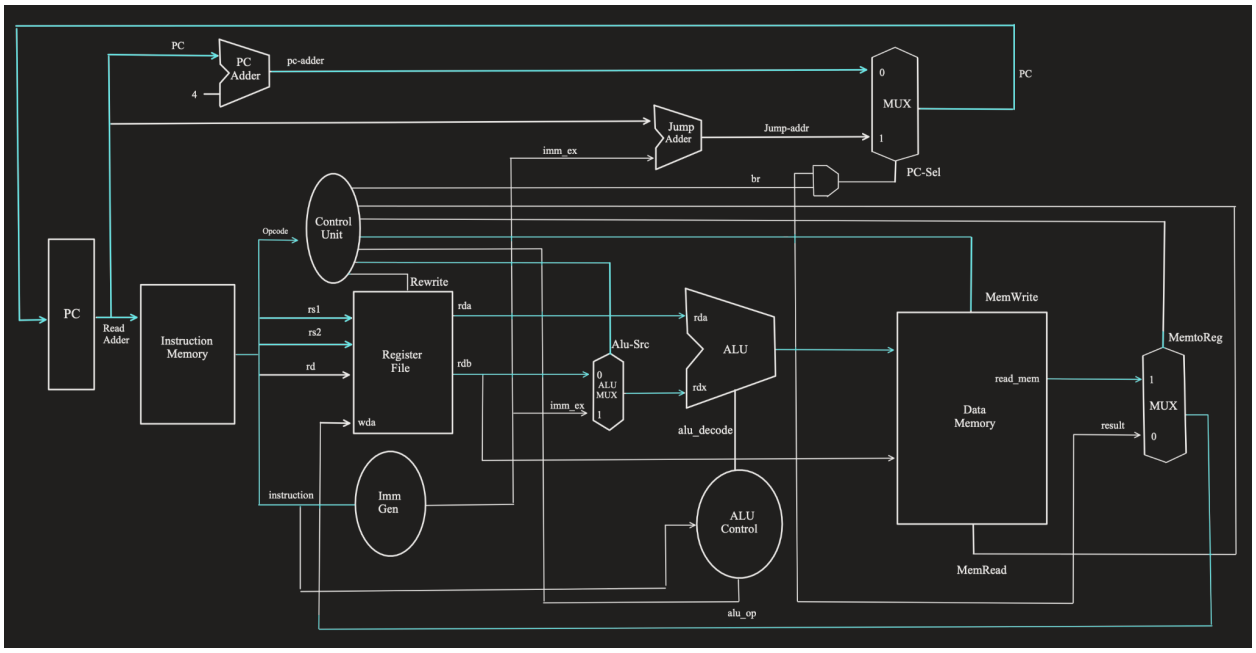


Figure 3: S-Type Datapath

Figure 3 depicts the datapath of an S-type instruction. These types of instructions are used to store data into a specific memory location. For this walkthrough, imagine a SW (store word) instruction. Similar to the above instruction types, the only difference at the start of the cycle is the derived control lines from the opcode. S-type instructions also contain an immediate which is decoded and sign extended in the immediate generation module. The following control bits are asserted in an S-type instruction: alu_src and mem_wrt as well as the 4-bit alu_op which is 0. Similar to the I-type, since alu_src is asserted, the decoded immediate is added to the contents of register rs1, which in the case of a store, is the null register, to compute the memory address to write to. The value that is written to memory is the value contained in the second register, rs2. Once the value is written, the instruction cycle is complete and the PC is incremented by 4 at the next positive edge.

B-type/J-type Datapath Diagram

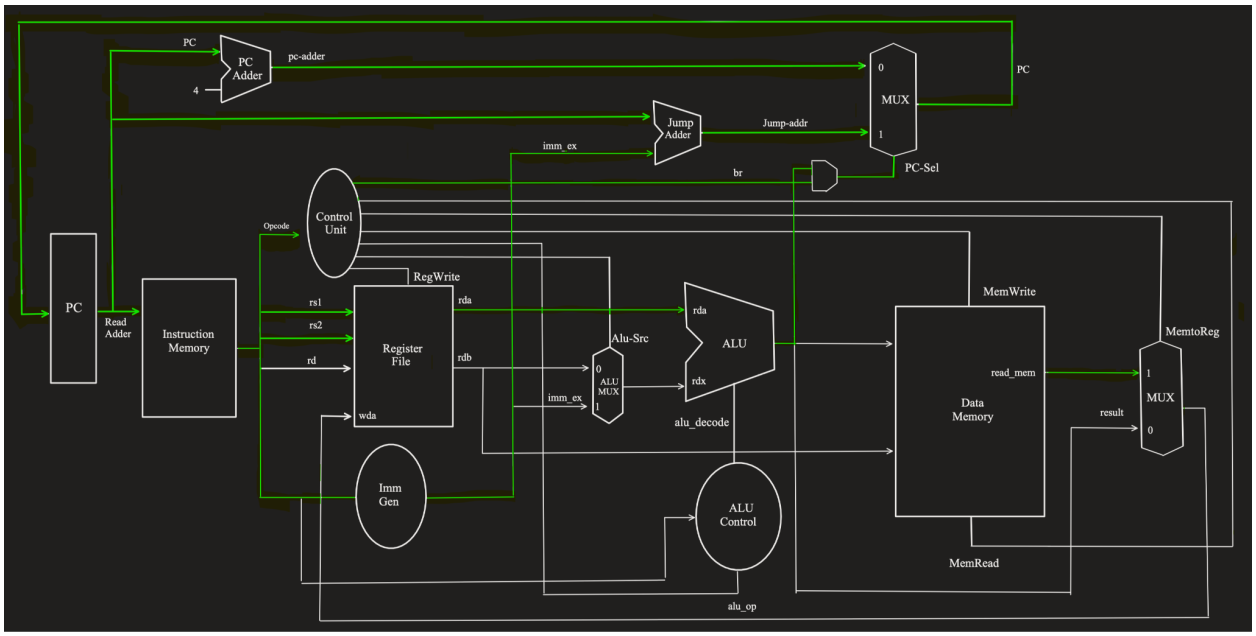


Figure 4: B-Type/J-Type Datapath

Figure 4 illustrates the datapath of a B-type/J-type instruction. These types of instructions are used to compare two registers and branch based on a specific condition. The branching "distance" is found in the encoded immediate value. The following control lines are asserted: branch (br) and the 4-bit alu_op. For this walkthrough, imagine a BEQ (branch if equal) instruction. The instruction format contains the two registers that are to be compared as well as an encoded immediate. Similar to the above instructions, the immediate is sign extended and decoded in the immediate generation module. The 4-bit alu_op, which is 7, tells the ALU to execute an XOR on the contents of rs1 and rs2 and if the result is 0, indicating the values are equal, increment the PC by the immediate rather than the usual 4. This allows the PC to jump to an instruction that is farther away from its current position.

Details on the Control Units

There are two control units in this CPU: the main control that decodes the opcode into its respective control lines and the second is the ALU control which uses information from the instruction as well as the derived alu_op value from the main control to tell the ALU which operation to execute. The verilog modules that represent these control units are the control.v and alu_ctrl.v files. The control.v has the input of the opcode and outputs all the control lines: alu_src, mem_to_reg, reg_wrt, mem_rd, mem_wrt, br, as well as the alu_op. The ALU control uses the alu_op as well as the funct7 and funct3 values from the instructions to determine the operation, producing the final alu_decode value. This alu_decode value is passed to the ALU which has logic to determine the operation to execute.

In the case of branching, which is a B-type instruction, the control unit derives the alu_op to be 7 (4'b0111) from the opcode. This value is then passed to the ALU control which uses it as a way to narrow down the possible operations. The ALU control then looks at the funct3 value from the B-type instruction and sees that it is 0 (3'b000) which means the alu_decode is 7. This value is sent to the ALU which executes the operation associated with the alu_decode value of 7, an XOR operation. If the register contents are equal, the zero flag is asserted and the immediate is added to the current PC in the jump adder module to achieve the new PC.

Test Program #1

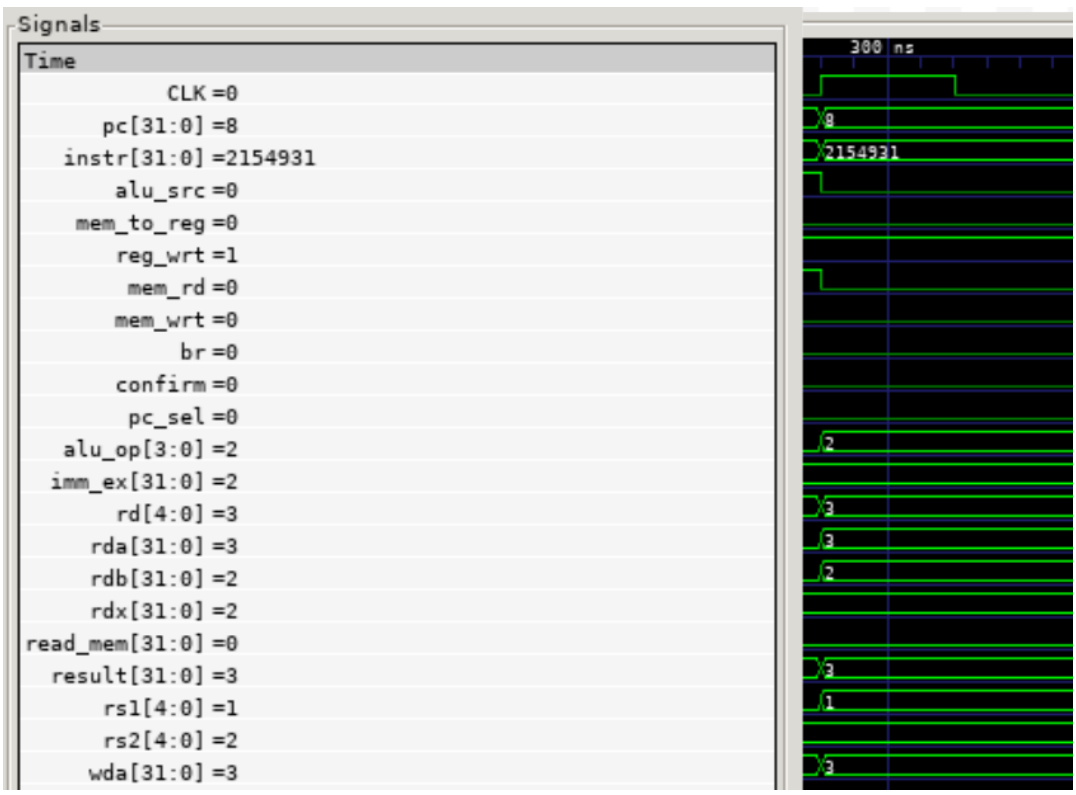
Below is one of two test programs that can be loaded into instruction memory and run on the CPU. To run this program, download all modules in "Final_Project_Modules" and open the instr_mem.v and cpu_tb.v file. Ensure the name of the text file that is being read in instr_mem.v is "Test_Program.txt". Then, compile cpu_tb.v and the program will run.

Assembly:	**Instruction Type**
ADDI x1, x0, #3	I-type
ADDI x2, x0, #2	I-type
OR x3, x1, x2	R-type
SW x2, 0(x0)	S-type
LW x4, 0(x0)	I-type
BEQ x2, x4, #16	B/J-type

As stated previously, this is an implementation of a single cycle CPU, meaning the Clock cycles Per Instruction (CPI) is 1. However, this clock cycle is the length of the time required for the slowest instruction (SW and LW since they require writing or reading from RAM). Below are the sections of the waveform generated from the Test Program depicting each of the specific instruction types.

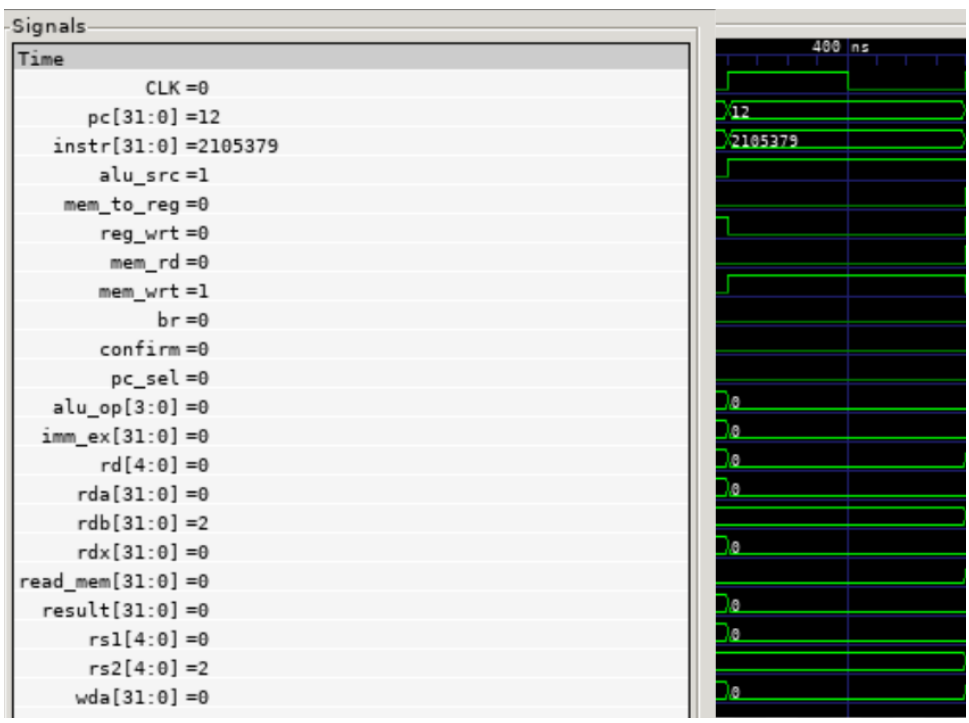
R-type:

This instruction is a bitwise OR between registers x1 and x2. The two instructions prior to this one were ADDI instructions to get the immediate values 3 and 2 into the registers x1 and x2, respectively. Logical operations are R-type instructions, therefore only the reg_wrt control line is asserted. When looking at the register values that enter the ALU, rda and rdx, the values are 2 and 3. The result[31:0] of the ALU is 3 because 2 | 3 when looked at in binary is 3. This value is then written to the destination register (rd[4:0]) x3.



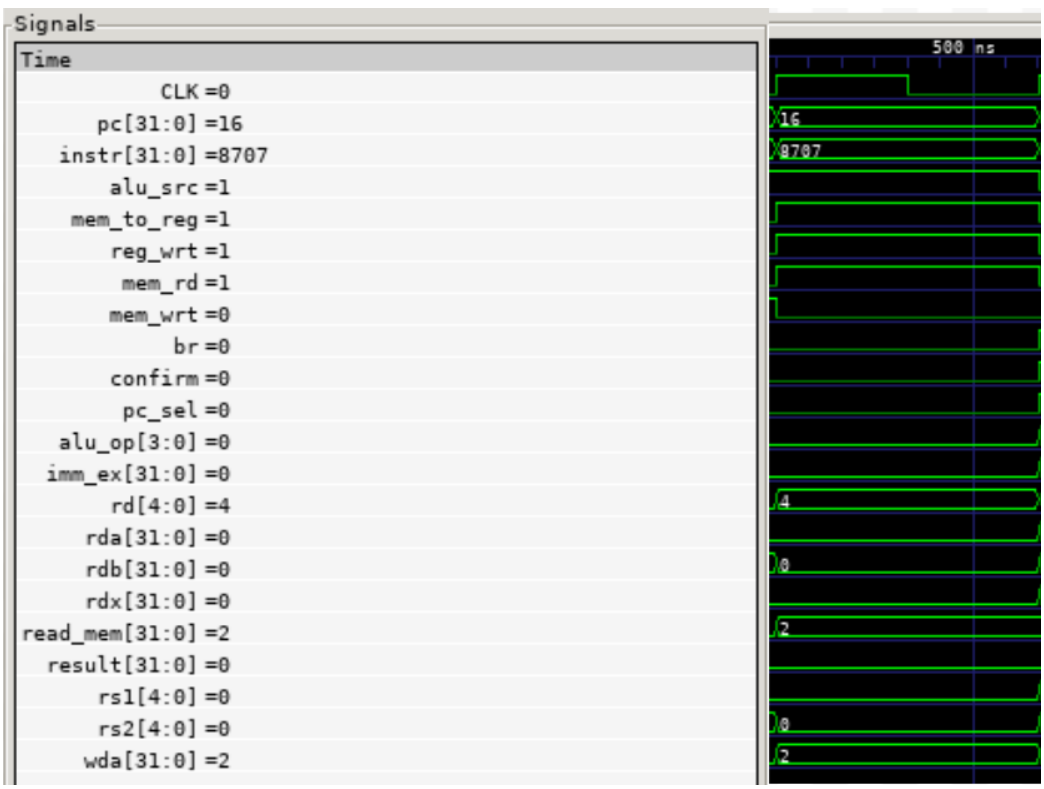
S-type:

This is the generated waveform for a SW instruction which is storing the contents of register x2 into the first memory address of data memory. SWs are S-type instructions which is why the alu_src and mem_wrt control lines are asserted. As seen in the assembly, the first instruction was ADDI x2, x0, #2 which is placing the immediate value 2 into register x2. Therefore, the value that was in register x2, #2, is being stored into the memory address that is computed by adding the offset, in this case 0 (imm_ex[31:0]), to the value stored in the null register. This means the content of register x2 is being written to the first memory address.



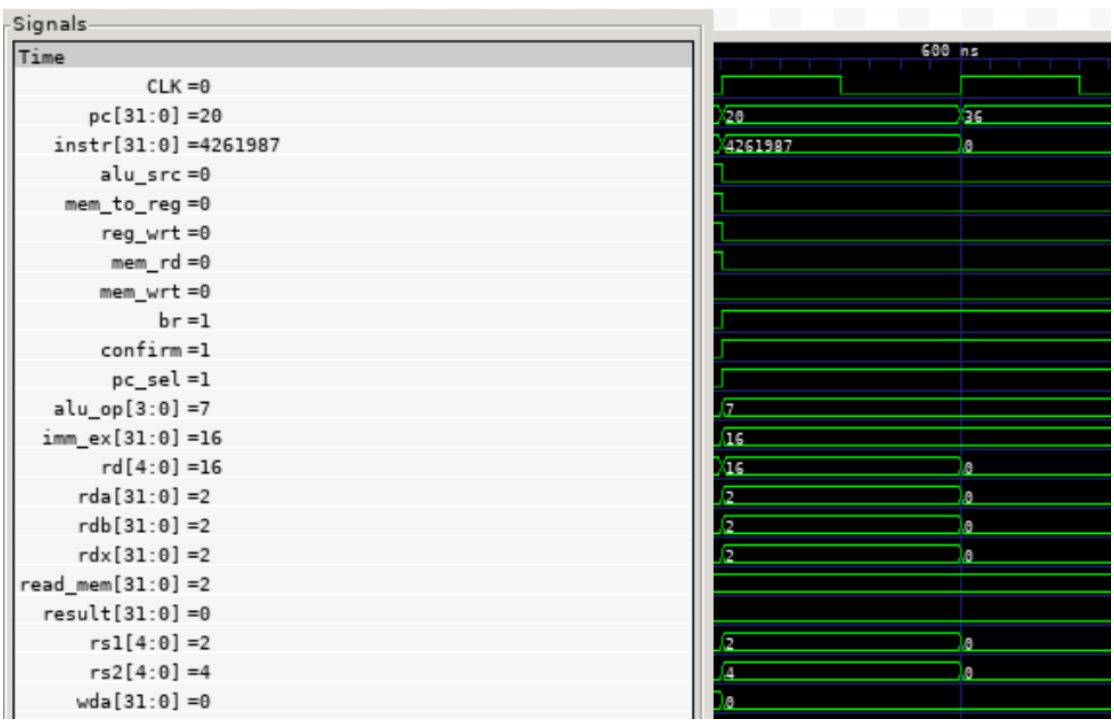
I-type:

This is the generated waveform for a LW instruction which is loading the value stored at the first address of data memory into register x4. As stated previously, LW is an I-type instruction which is why alu_src, mem_to_reg, reg_wrt, and mem_rd are asserted. Furthermore, the assembly code shows that the register x4 is being written to which can be seen in the waveform where rd[4:0] is the value 4. Finally, the instruction before this load instruction was a store instruction of the value 2 into memory. This is why the read_mem[31:0] value is a 2 because the stored value is being read from memory and written back to the register file.



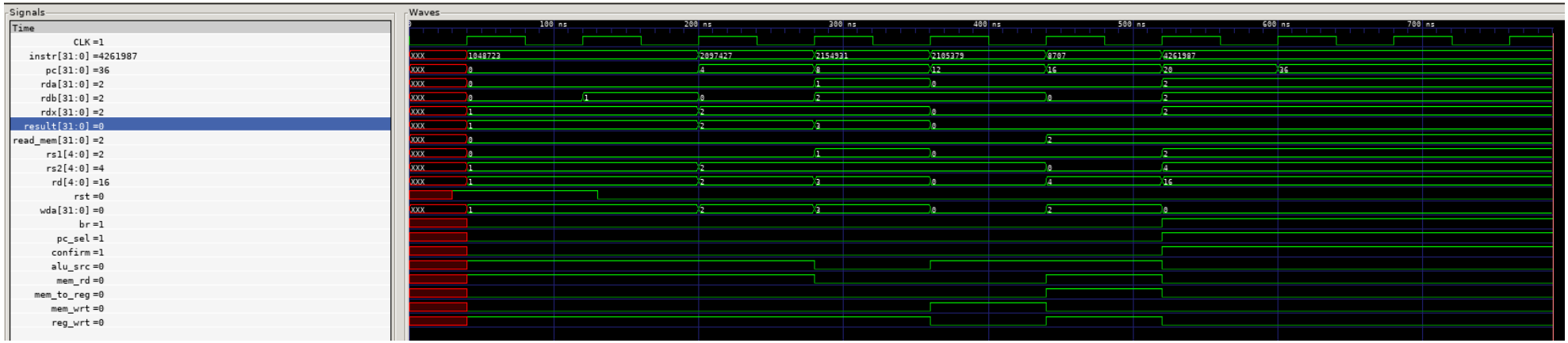
B-type:

This instruction is the waveform generated by BEQ instruction which compares the contents of registers x2 and x4 and branches to a new instruction address achieved by adding the immediate (#16) to the current program counter. BEQ is a B-type instruction which is why the br control line is asserted. The value that loaded int x4, #2, is the same value that was initially stored into x2 with the ADDI instruction. Therefore, because both x2 and x4 contain the same value, when they are XORed together, the resulting value is 0. This will assert the zero flag (confirm signal) which is ANDed with the br control line to assert the pc_sel control. This takes the encoded immediate value, #16, and adds it to the current PC, 20, to get the final PC of 36, which is seen on the next rising edge of the clock cycle.



Full waveform of Test Program #1

This is the entire waveform of Test Program #1 that was analyzed in the previous section.

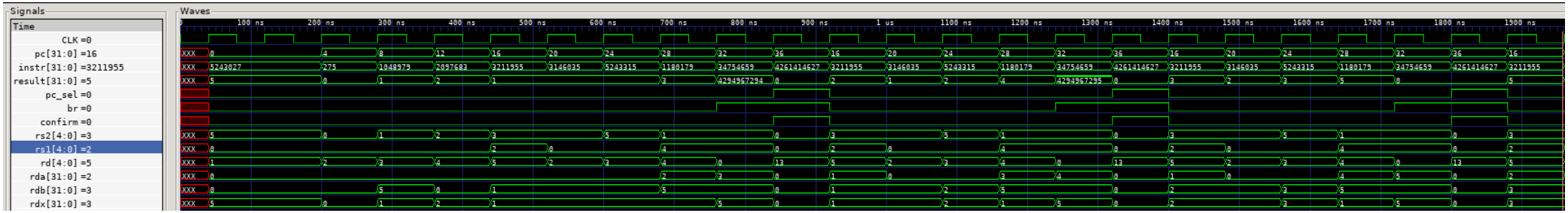


Test Program #2

The second test program that can be run on the CPU is the code for an iterative fibonacci sequence. Below is the assembly code for this program which uses many of the same instructions as the test program, however it involves a for loop. To run this program, open the intr_mem.v file and change the name of the text file that is being read from "Test_Program.txt" to "Fibonacci_Sequence.txt". Then, run cpu_tb.v to see the fibonacci sequence.

PC:	**Assembly**	**Instruction Type**
0	ADDI x1, x0, #5	I-type
4	ADDI x2, x0, #0	I-type
8	ADDI x3, x0, #1	I-type
12	ADDI x4, x0, #2	I-type
	Loop:	
16	ADD x5, x2, x3	R-type
20	ADD x2, x0, x3	R-type
24	ADD x3, x0, x5	R-type
28	ADDI x4, x4, #1	I-type
32	B. Loop	B-type

This is the waveform generated by the fibonacci sequence program. If you look at the result[31:0] value under every instance where the pc[31:0] singal is 16, you will see the fibonacci sequence. This is because the 5th instruction (red text) is the accumulator for the sums in the fibonacci sequence. This program was written to determine the fibonacci sequence up to n=5.



Registers x2 and x3 initialized to #0 and #1, respectively.

Outside of Loop: Instruction PC: 4, Result: 0

Outside of Loop: Instruction PC: 8, Result: 1

Loop 1: Instruction PC 16, Result: 1

Loop 2: Instruction PC: 16, Result: 2

Loop 3: Instruction PC: 16, Result: 3

Loop 4: Instruction PC: 16, Result: 5